Estudiantes:
Axel Javier Rosas Rodríguez | A01738607
Elian Cantalapiedra Sabugal | A01738462
Edwin Emmanuel Salazar Meza | A01738380

Profesor:
Rigoberto Cerino Jiménez

Nombre de la Institución:
Instituto Tecnológico y de Estudios Superiores de Tecnológico de Monterrey

Materia:
Fundamentación de la Robótica

Fecha:
03 de Marzo de 2026

Título del trabajo:
Actividad 2.2 PWM con ESP32

Resumen

Esta actividad práctica tiene como objetivo implementar un control de velocidad y sentido de giro de un motor de corriente continua utilizando un esp32, mediante una salida de señal pwm y una entrada de un potenciómetro. El potenciómetro permite variar la potencia del motor a través del ciclo de pwm, además de poder controlar el giro por medio del puente H.

La solución a esta actividad está basada en las distintas configuraciones de los pines de entrada y salida del esp32, este sistema se comunica con una integración de micro-ROS, permitiendo mostrar información del motor en una computadora.

Objetivos

Objetivo General:

- Implementar un sistema de control de velocidad y sentido de giro de un motor de corriente continua utilizando un esp32 mediante modulación por ancho de pulso (PWM), integrando micro-ROS para la comunicación y monitoreo de variables como voltaje, duty cycle y estado del motor, a partir de la lectura de un potenciómetro.

Objetivos específicos:

- Configurar el ESP32 para la generación de señales PWM con una frecuencia y resolución adecuadas.
- Implementar la lectura de un potenciómetro para ajustar el duty cycle del PWM.
- Controlar el sentido de giro del motor DC mediante pines digitales.
- Integrar micro-ROS para la comunicación entre el ESP32 y el sistema ROS.
- Publicar y suscribirse a tópicos para monitorear voltaje, duty cycle y estado del motor.
- Verificar el funcionamiento del sistema mediante la observación de gráficas y el comportamiento del motor.

Introducción

En esta práctica se aborda el control de un motor de corriente directa mediante un esp32, combinando técnicas de control a bajo nivel con herramientas de comunicación propias de sistemas robóticos modernos. El objetivo principal es demostrar cómo un microcontrolador puede integrarse de forma efectiva con micro-ros para permitir el control, supervisión y envío de comandos desde una computadora o dispositivo externo. Para ello, se emplea la modulación por ancho de pulso (PWM) como método para regular la velocidad del motor y se incorporan mecanismos que aseguren una comunicación confiable entre el microcontrolador y el sistema de alto nivel.

Micro-ROS es una extensión de ROS 2 diseñada específicamente para microcontroladores con recursos limitados como el esp32. Su funcionamiento se basa en un modelo cliente–agente: el microcontrolador ejecuta un cliente Micro-ROS que se comunica con un agente Micro-ROS corriendo en una computadora o sistema más potente. A través de esta

comunicación, el microcontrolador puede publicar y suscribirse a tópicos de ROS 2, permitiendo el envío de comandos y la recepción de datos en tiempo real. Esto posibilita que un motor sea controlado desde ROS 2 de manera estructurada y estandarizada.

Ante el uso de un microcontrolador externo implica que la comunicación entre el microcontrolador y el agente Micro-ROS puede verse afectada por fallos de conexión, es necesario implementar mecanismos que garanticen el funcionamiento constante del sistema. Para ello, se utiliza un sistema de reconexión basado en una máquina de estados, proporcionado por el socio formador Manchester Robotics. Este enfoque divide el funcionamiento del sistema en distintos estados, como espera de conexión, conexión activa y desconexión. La máquina de estados permite detectar cuándo el agente está disponible, crear o destruir dinámicamente las entidades de Micro-ROS y asegurar que el sistema pueda recuperarse automáticamente ante una pérdida de comunicación, sin necesidad de reiniciar el microcontrolador manualmente

El control de la velocidad del motor se realiza mediante PWM (Pulse Width Modulation) en el esp32. El pwm es una técnica que permite regular la potencia entregada a un motor variando el ciclo de trabajo de una señal digital, es decir, el porcentaje de tiempo que la señal permanece en nivel alto dentro de un periodo fijo. Aunque el voltaje máximo aplicado al motor se mantiene constante, el voltaje promedio depende del ciclo de trabajo, lo que se traduce directamente en cambios de velocidad.

El puente H es un circuito electrónico utilizado para controlar el sentido de giro de un motor de corriente directa, permitiendo invertir la polaridad aplicada a sus terminales. Su funcionamiento se basa en un arreglo de interruptores electrónicos (transistores) que, al activarse en distintas combinaciones, hacen que el motor gire en sentido horario o antihorario, o se detenga.

En esta práctica, el puente H actúa como la etapa de potencia entre el ESP32 y el motor, ya que el microcontrolador no puede suministrar directamente la corriente necesaria para accionarlo. El esp envía señales digitales para definir la dirección de giro y una señal pwm para controlar la velocidad, mientras que el puente H se encarga de manejar la energía requerida por el motor de forma segura y eficiente. Esto permite un control completo del motor tanto en velocidad como en dirección desde el sistema embebido y su integración con micro-ROS.

El esp32 cuenta con módulos de hardware dedicados para generar señales PWM con alta precisión, permitiendo configurar parámetros como la frecuencia y la resolución para adaptarse a las necesidades del sistema.

Metodología

Para realizar el sistema, los componentes necesarios para el control del motor fueron una ESP32 como unidad de procesamiento, un motor de corriente directa, un puente H para el control de potencia y sentido de giro, y un potenciómetro para la entrada analógica que permite variar la velocidad del motor. Asimismo, se definió la necesidad de integrar el sistema con micro-ROS para la comunicación y monitoreo desde una computadora.

Inicialmente, se identificaron y configuraron los pines del ESP32 para la lectura del potenciómetro, la generación de la señal PWM y el control del sentido de giro del motor mediante un puente H. Posteriormente, se establecieron los parámetros del PWM con una frecuencia de 5000 Hz y una resolución de 8 bits, permitiendo regular la velocidad del motor a partir del duty cycle calculado con la señal analógica del potenciómetro.

De manera paralela, se integró micro-ROS mediante la creación de un nodo, publishers y un subscriber, lo que permitió la comunicación entre el ESP32 y el sistema ROS 2. A través del subscriber se interpretaron los comandos recibidos para definir el estado y la dirección de giro del motor.

Finalmente, se validó el funcionamiento del sistema verificando la correcta publicación de los tópicos de voltaje, duty cycle y estado del motor, así como el comportamiento físico del motor ante distintos comandos y valores de entrada.

Solución del problema

```
#define VAR_PIN 14
#define IN1_PIN 25
#define IN2_PIN 26
#define PWM_PIN 27

#define PWM_FRQ 5000 // Hz
#define PWM_RES 8 // Bits 0-255
#define PWM_CHNL 0
```

Se comienza definiendo los pines a usar del ESP32 para el manejo del motor. Un pin para la lectura del potenciómetro, dos para la dirección de giro y un pin para la señal PWM.

Se establecen los parámetros del PWM con una frecuencia de 5000 Hz y una resolución de 8 bits.

```
// Micro-ROS Entity Declarations
rclc_support_t support;        // Micr
rclc_executor_t executor;      // Mana
rcl_allocator_t allocator;     // Hand

rcl_node_t node;               // Defi

rcl_subscription_t command_sub;
```

```
rcl_publisher_t duty_pub;
rcl_publisher_t voltaje_pub;
rcl_publisher_t state_pub;

std_msgs__msg__String command_msg;
std_msgs__msg__Float32 duty_msg;
std_msgs__msg__Float32 voltaje_msg;
std_msgs__msg__String state_msg;
```

Hacemos la declaración de entidades de micro-ROS, entre ellos se encuentra el nodo, suscriptor y publishers. Así mismo, hacemos la definición de los tipos de datos que usaremos, dos strings y dos float32.

```
void subscription_callback(const void *msgin)
{
    std_msgs__msg__String *msg = (std_msgs__msg__String *)msgin;
    char opcion = msg->data.data[0];

    switch (opcion) {
        case 'D':
            motorEnMovimiento = true;
            derecha();
            break;
        case 'I':
            motorEnMovimiento = true;
            izquierda();
            break;
        case 'S':
            motorEnMovimiento = false;
            detener();
            break;
        default:
            break; // Comando no válido
```

Definimos la respuesta del suscriptor dependiendo del tipo de dato publicado, en este escenario D es para un giro a la derecha, I para un giro a la izquierda y S para detener el movimiento.

En caso de que el dato recibido por el suscriptor no entre en ninguno de los 3 casos, no ejecuta ninguna operación hasta recibir un dato válido.

```
void setup()
{
  set_microros_transports();

  pinMode(VAR_PIN, INPUT);
  pinMode(IN1_PIN, OUTPUT);
  pinMode(IN2_PIN, OUTPUT);

  ledcSetup(PWM_CHNL, PWM_FRQ, PWM_RES);
  ledcAttachPin(PWM_PIN, PWM_CHNL);

  detener();
```

En el setup comenzamos por establecer la comunicación con micro-ROS mediante la función set_microros_transports().

Posteriormente inicializamos y definimos los pines a usar, como entrada en el caso del potenciómetro y como salida para los pines que controlan el giro del motor. También definimos los parámetros del PWM.

```
// Publishers
rclc_publisher_init_default(
  &duty_pub,
  &node,
  ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32),
  "/duty_cycle");

rclc_publisher_init_default(
  &voltaje_pub,
  &node,
  ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32),
  "/voltaje");
```

```
rclc_publisher_init_default(
  &state_pub,
  &node,
  ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, String),
  "/motor_state");

// Subscriber
rclc_subscription_init_default(
  &command_sub,
  &node,
  ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, String),
  "/motor_command");
```

Dentro del mismo setup, creamos los tópicos a usar, en este caso son 3 publishers y un subscriber, pasando como argumentos el nodo, la variable donde guardará el dato y el tipo de mensaje a usar.

```
void loop()
{
  rclc_executor_spin_some(&executor, RCL_MS_TO_NS(10));

  if (motorEnMovimiento) {
    leerPWM();
    duty_msg.data = duty;
    voltaje_msg.data = voltaje;

    rcl_publish(&duty_pub, &duty_msg, NULL);
    rcl_publish(&voltaje_pub, &voltaje_msg, NULL);
  }

  state_msg.data.size = strlen(state_buffer);
  rcl_publish(&state_pub, &state_msg, NULL);

  delay(100);
```

Dentro del loop definimos un ejecutor que verifica si existen mensajes entrantes, permitiendo que el nodo procese comandos recibidos sin bloquear el programa.

por otra parte hacemos consulta de la variable motorEnMoviminento, para que solo publique la información de los topicos /duty_cycle y /voltaje si es que el motor está girando.

Finalmente, en una función auxiliar hacemos la corrección al escalado de la lectura analógica. Dado que el ADC del ESP32 posee una resolución de 12 bits (rango 0–4095), utilizar un valor de 1023 (10 bits) generaba un error en la conversión, produciendo una aparente saturación o reinicio en la lectura al alcanzar aproximadamente una cuarta parte del recorrido del potenciómetro.

```
void leerPWM()
{
  pot = analogRead(VAR_PIN);

  voltaje = pot * (3.3 / 4095.0);
  duty = (voltaje / 3.3) * 100.0;

  pwm = map(pot, 0, 4095, 0, 255);
  ledcWrite(PWM_CHNL, pwm);
}
```

Al ajustar el valor a 4095, se eliminó dicha inconsistencia y se obtuvo una lectura correcta en todo el rango de operación del potenciometro.

Resultados

Se muestra una correcta inicialización del agente de micro-ROS, así como la creación exitosa de los publishers y el suscriber.

```
reiv@DinoRex-HP-Victus:~$ ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB
0
[1772576819.484641] info     | TermiosAgentLinux.cpp | init                     | running..
.     | fd: 3
[1772576819.485343] info     | Root.cpp              | set_verbose_level        | logger setup
      | verbose_level: 4
[1772576822.606647] info     | Root.cpp              | create_client            | create
      | client_key: 0x5E0BD9A7, session_id: 0x81
[1772576822.606762] info     | SessionManager.hpp    | establish_session        | session esta
blished   | client_key: 0x5E0BD9A7, address: 0
[1772576822.635289] info     | ProxyClient.cpp       | create_participant       | participant
created   | client_key: 0x5E0BD9A7, participant_id: 0x000(1)
[1772576822.653812] info     | ProxyClient.cpp       | create_topic             | topic create
d         | client_key: 0x5E0BD9A7, topic_id: 0x000(2), participant_id: 0x000(1)
[1772576822.667090] info     | ProxyClient.cpp       | create_publisher         | publisher cr
eated     | client_key: 0x5E0BD9A7, publisher_id: 0x000(3), participant_id: 0x000(1)
[1772576822.681885] info     | ProxyClient.cpp       | create_datawriter        | datawriter c
reated    | client_key: 0x5E0BD9A7, datawriter_id: 0x000(5), publisher_id: 0x000(3)
[1772576822.699253] info     | ProxyClient.cpp       | create_topic             | topic create
d         | client_key: 0x5E0BD9A7, topic_id: 0x001(2), participant_id: 0x000(1)
[1772576822.711784] info     | ProxyClient.cpp       | create_publisher         | publisher cr
eated     | client_key: 0x5E0BD9A7, publisher_id: 0x001(3), participant_id: 0x000(1)
[1772576822.725929] info     | ProxyClient.cpp       | create_datawriter        | datawriter c
reated    | client_key: 0x5E0BD9A7, datawriter_id: 0x001(5), publisher_id: 0x001(3)
[1772576822.742829] info     | ProxyClient.cpp       | create_topic             | topic create
d         | client_key: 0x5E0BD9A7, topic_id: 0x002(2), participant_id: 0x000(1)
[1772576822.755783] info     | ProxyClient.cpp       | create_publisher         | publisher cr
eated     | client_key: 0x5E0BD9A7, publisher_id: 0x002(3), participant_id: 0x000(1)
[1772576822.771911] info     | ProxyClient.cpp       | create_datawriter        | datawriter c
reated    | client_key: 0x5E0BD9A7, datawriter_id: 0x002(5), publisher_id: 0x002(3)
[1772576822.789438] info     | ProxyClient.cpp       | create_topic             | topic create
d         | client_key: 0x5E0BD9A7, topic_id: 0x003(2), participant_id: 0x000(1)
[1772576822.803673] info     | ProxyClient.cpp       | create_subscriber        | subscriber c
reated    | client_key: 0x5E0BD9A7, subscriber_id: 0x000(4), participant_id: 0x000(1)
[1772576822.819401] info     | ProxyClient.cpp       | create_datareader        | datareader c
reated    | client_key: 0x5E0BD9A7, datareader_id: 0x000(6), subscriber_id: 0x000(4)
```

Se muestra correctamente el cambio de dirección del motor, así como la publicación de los tópicos de /voltaje y /duty_cycle.

Tópico /command_sub

```
publishing #26: std_msgs.msg.String(data='D')

publishing #27: std_msgs.msg.String(data='D')

publishing #28: std_msgs.msg.String(data='D')
```

Tópico /voltaje

```
---
data: 3.299999952316284
---
data: 3.299999952316284
---
data: 3.299999952316284
---
```

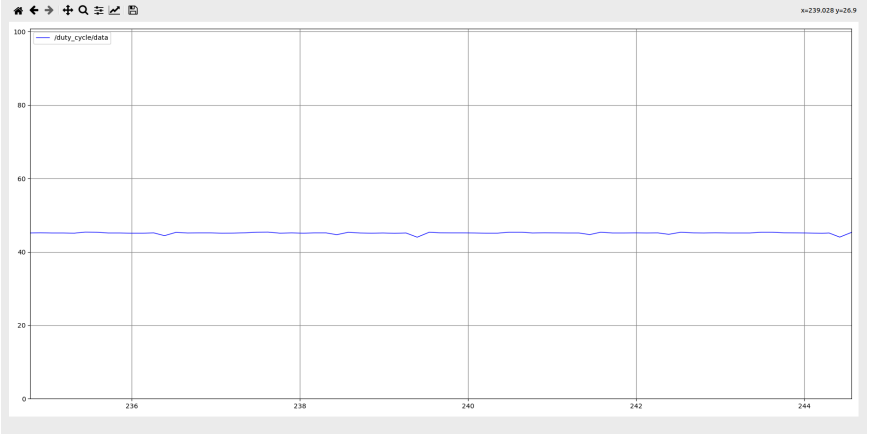
Tópico /duty_cycle

```
---
data: 100.0
---
data: 100.0
---
data: 100.0
---
```

Tópico /motor_state

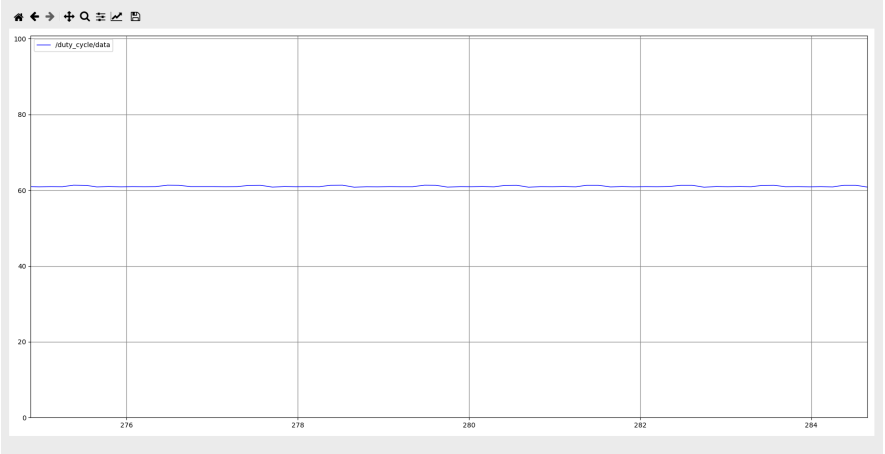
```
data: Derecha
---
data: Derecha
---
data: Derecha
---
data: Derecha
```

Gráfica del duty_cycle/data



Como se puede apreciar en las gráficas, el duty cycle se mantiene casi constante tras definirse su valor con el potenciómetro, presentando pequeñas oscilaciones en la señal de voltaje entregada .

Este comportamiento puede atribuirse al desgaste del mismo potenciómetro, lo cual generaría variaciones en la resistencia y, por consiguiente, inestabilidad en la señal de salida.



Aun así se considera el manejo con micro-ros efectivo, pudiendo controlar el sentido y la velocidad de giro de un motor a través de un ESP32.

Conclusiones

El desarrollo de esta práctica permitió evidenciar que el control de un motor de corriente directa con el ESP32 y PWM es una solución funcional y flexible; sin embargo, también dejó en claro que el desempeño del sistema depende en gran medida de una correcta comprensión del hardware involucrado. Aspectos como la resolución real del ADC o las limitaciones físicas de componentes sencillos, como el potenciómetro, pueden introducir errores o variaciones que afectan directamente la estabilidad del control si no se consideran adecuadamente.

La integración de micro-ROS resultó valiosa ya que nos facilitó la supervisión y el control del sistema desde un entorno de alto nivel, reforzando la importancia de la comunicación estructurada en aplicaciones robóticas modernas. No obstante, esta integración también pone de manifiesto la necesidad de diseñar sistemas robustos ante fallos de comunicación y de comprender el impacto que estos pueden tener en el comportamiento del actuador.

En conjunto, la práctica no solo cumplió con el objetivo, sino que permitió reflexionar sobre la importancia de validar las suposiciones de diseño, comprender las características reales del hardware y considerar la interacción entre sistemas embebidos y plataformas robóticas de mayor nivel para lograr soluciones confiables y escalables.

Referencias

Features and Architecture. (2025, September 16). Micro-ROS. https://micro.ros.org/docs/overview/features/

PWM of ESP32 | ESP32. (2019). Electronicwings.com. https://www.electronicwings.com/esp32/pwm-of-esp32